* note: the formatting is different between my 2 annotated traces due to having to switch PDF export tools after hitting the page limit.

You asked:

The following is a coding exercise called "Inductive Bias of CNNs" for a deep learning class. You are an expert in deep learning and Python programming. I am uploading the .ipynb file for you to read, and will then paste in the individual sections of code/questions for you to answer. Here is the ipynb file.

I uploaded the .ipynb file fully, with very little explanation, at the beginning of the conversation so that it would be in context. I wanted to evaluate whether Gemini would be able to recall the contents throughout the rest of the long conversation.

Gemini Replied:

I have analyzed the `edge_detection.ipynb` file you uploaded. It appears to be a comprehensive assignment focused on exploring the **inductive bias** of Convolutional Neural Networks (CNNs) compared to Multi-Layer Perceptrons (MLPs) using a synthetic edge detection task.

I am ready to help you with the specific sections. Based on the notebook, I have context on the following components that will likely be relevant to your questions:

- **`EdgeDetectionDataset`**: A custom class for generating synthetic images (28x28) with horizontal, vertical, diagonal, or no edges, including parameters for controlling edge location, intensity, and background noise.
- **Models**: Definitions for `ThreeLayerCNN` (likely referenced as `SimpleCNN` in parts of the code) and `TwoLayerMLP` / `ThreeLayerMLP`.
- **Experiments**:
  - **Data Efficiency**: Comparing CNN and MLP performance on small datasets (e.g., 10 images per class).
  - **Translation Invariance**: Testing how models handle edges in different locations (domain shift between train/validation).
  - **Inductive Bias Failure**: Experiments using **random pixel permutation** to break the local spatial structure, theoretically hurting CNNs more than MLPs.
  - **Hyperparameter Tuning**: "TODO" sections for adjusting learning rates, epochs, and optimizers (SGD vs. Adam).

Please paste the first section of code or the specific question you would like me to address!

You asked:

Q1. Overfitting Models to Small Dataset

In this problem, we will make our models overfit the small dataset to test the model architecture and our synthetic dataset. We use the same dataset for both models. Let's generate a small dataset with

ten images per class.

set_seed(seed)

small_dataset_config = None

small_dataset = None

transforms = T.Compose([T.ToTensor()])

##

TODO: Generate dataset with 10 images per class

Hint: Refer visualize_data_config and use EdgeDetectionDataset and

transforms function provided above.

##

##

END OF YOUR CODE

##

Gemini Replied:

Based on the "10 images per class" requirement and the typical structure of the `EdgeDetectionDataset` used in this assignment, here is the code to fill in the missing section.

I am assuming standard parameters (`img_size=28`, standard noise levels) and that the dataset calculates the total number of samples based on the classes, or simply accepts a total count (4 classes × 10 images = 40 total images).

small_dataset_generation.py
Dec 3, 4:30 PM

Note on parameters:

- **`num_samples`**: I set this to **40** (10 images × 4 classes). If your `EdgeDetectionDataset` logic automatically multiplies a "per_class" number by the number of classes, you might need to change this to 10. However, usually, this parameter defines the *total* size of the dataset.
- **`img_size`**: Set to 28 to match standard inputs (like MNIST) and the likely model architecture.

You asked:

small_dataset_loader = None

##

TODO: Implement dataloader

Hint: You should flag shuffle = True for training data loader

This flag makes huge difference in training

##

##

END OF YOUR CODE

##

Gemini Replied:

I've added the `DataLoader` implementation to your file. I set the `batch_size` to 40 (the size of the entire small dataset) to allow for efficient overfitting, and ensured `shuffle=True` as requested.

At first, the batch size was set to 40. I was unsure of why as this directly contradicted the instructions, so I prompted Gemini to attempt to match the desired formatting.

You asked:

Does the code you provided for small_dataset_config ensure that there are 10 samples per class and 3 classes (horizontal, vertical, and none)?

Gemini Replied:

It appears I assumed 4 classes (including a "Diagonal" class) in my previous response, which is common in similar datasets.

If the dataset specifically uses **3 classes** (Horizontal, Vertical, and None), then setting `num_samples` to 40 would result in approximately 13 images per class ($40/3$), not 10.

To ensure exactly **10 samples per class** with **3 classes**, `num_samples` must be set to **30**. I have updated the Canvas to reflect this correction.

This is interesting: Gemini thinks there is a "Diagonal" class, either due to having other similar problems in its training or due to seeing the diagonal class from the .ipynb file later in this exercise.

You asked:

I need to have exactly 10 samples per image. Can you achieve this by setting data_per_class, num_classes, and class_type in the dict?

Gemini Replied:

Yes, if your `EdgeDetectionDataset` implementation supports those specific keys, that is a much more robust way to define the dataset balance. Using `data_per_class` ensures exactly 10 images for each of the 3 specified types, rather than relying on a random split of a total `num_samples`.

Here is the updated code using the keys you requested: `data_per_class`, `num_classes`, and `class_type`.

All code is included at the end of this document.

The code ran correctly and was able to train (one-shot)!

You asked:

Question

Can you find any interesting patterns in the learned filters? Answer this question in your submission of the written assignment.

For this problem, I did not explicitely provide the images, but in future similar parts, I did. Here it appears to infer the context even without the screenshot.

Gemini Replied:

Based on the context of this specific edge detection exercise, you typically look for the following patterns in the first-layer filters of the CNN:

Expected Patterns:

1. **Directional Edge Detectors:** You will likely see filters that look like **Sobel** or **Prewitt** operators.
    - **Vertical Edge Filters:** These often appear as a vertical band of light pixels (positive weights) next to a vertical band of dark pixels (negative weights). This pattern detects changes in intensity moving horizontally across the image.
    - **Horizontal Edge Filters:** These often appear as a horizontal band of light pixels stacked on top of or below a band of dark pixels. This detects intensity changes moving vertically.
2. **Sparsity/Noise:**
    - Since the task is synthetic and very clean, the filters are often quite sharp and interpretable compared to filters learned on complex datasets like ImageNet.
    - However, if the model is overfitting or the initialization was unlucky, some filters might look like random noise or "dead" filters (mostly gray) if they weren't utilized during training.

Why this happens: These patterns emerge because the CNN's **inductive bias** (specifically locality and translation invariance) encourages it to learn small, local features that are useful across the entire image. Since the classes are defined entirely by the presence of vertical or horizontal edges,

the most efficient way to minimize loss is to learn filters that specifically "fire" (produce high activation) when they convolve over those specific edge orientations.

> **You asked:**

Q2. Sweeping the Number of Training Images

We understood the given task and checked that both models had enough expressive power. We will compare the performance of MLP and CNN by changing the number of data per class. We expect that the model with proper inductive biases on this task will fit with fewer training examples. And let's see which one has inductive biases. In this problem, we will use the same dataset for both models. We sweep the number of training images from 10 to 500. The validation set will be the same for all the experiments.

```
set_seed(seed)


train_loader_dict = dict()

num_images_list = [10, 30, 50, 100]

valid_loader = None


transforms = T.Compose([T.ToTensor()])

train_batch_size = 10

valid_batch_size = 256
################################################################################
# TODO: Implement train_loader_dict for each number of training images.     #
# Key: The number of training images (5, 10, 30, 50, and 100)               #
# Value: The corresponding dataloader                                       #
# The validation set size is 50 images per class                            #
################################################################################
################################################################################
#                             END OF YOUR CODE                              #
################################################################################
```

Gemini Replied:

Here is the code to implement the data sweep.

I have created a dictionary for the training loaders corresponding to the number of images specified in `num_images_list`, and a single validation loader with 50 images per class.

Dataset Sweep
Dec 3, 4:42 PM

Type text here

I had to prompt again to obtain the desired set up for the data loader based on the instructions.

You asked:

Use only data_per_class in the dataset config and call EdgeDetectionDataset(config, 'train' or 'valid', transform=transforms) exactly, with no additional parameters.

Gemini Replied:

I've updated the `dataset_sweep.py` file. I have modified the configuration to rely solely on `data_per_class` for sizing (removing `num_classes` and `class_type`) and updated the `EdgeDetectionDataset` instantiation to strictly follow the signature `(config, mode, transform=transforms)` as requested.

The above version of the code is the final, attatched version, and worked correctly. While getting the right code itself required some back-and-forth, the code always ran correctly even if the loaded did not get files as requested.

You asked:

Question

Compare the learned kernels, untrainable kernels, and edge-detector kernels. What do you observe? Answer this question in your submission of the written assignment.

Gemini Replied:

Based on the context of this edge detection experiment, here is the expected observation for your written assignment:

Observations on Kernel Comparison:

1. **Learned Kernels:**
    - The kernels learned by the trained CNN will likely evolve to look very similar to standard edge detection filters (like Sobel or Prewitt operators).
    - You should observe distinct horizontal and vertical bands of positive and negative weights (light and dark rows/columns).

- This indicates the model has successfully "discovered" that identifying gradients in specific directions is the optimal way to solve this task.

2. **Untrainable (Random) Kernels:**
   - These kernels will appear as unstructured, random noise (speckled patterns of gray, white, and black) because they are typically initialized randomly and not updated.
   - They lack any geometric coherence and fail to extract meaningful features like "edges" efficiently.
3. **Edge-Detector Kernels (Hardcoded/Ideal):**
   - These are the "cleanest" versions, showing perfect mathematical representations of derivatives (e.g., a row of -1s on top of a row of 1s).

Conclusion: The **Learned Kernels** should closely resemble the **Edge-Detector Kernels**, proving that the CNN is capable of recovering the true, optimal features for the task purely from data. In contrast, the difference between the Learned and Untrainable kernels highlights the importance of feature learning (or proper initialization) for achieving high performance.

The above answer is reasonable and provides
a good explanation based on the context of the problem.

> **You asked:**

> **Gemini Replied:**

I see you have selected the code for the dataset sweep. It appears you are reviewing the implementation for generating the training loaders with varying dataset sizes.

The logic in the selected code correctly iterates through `num_images_list` to create specific loaders for each size (10, 30, 50, 100) while maintaining a consistent 50-image-per-class validation set.

I have updated the **Canvas** file to include the initialization variables (`num_images_list` , `transforms` , etc.) and necessary imports. This makes `dataset_sweep.py` a complete, self-contained script that you can reference or run directly, rather than just the snippet inside the loop.

This version is now self-contained. If you are ready for the next step (training the MLP and CNN models on these datasets), please let me know or paste the next section of the notebook!

> **You asked:**

Question

We freeze the convolutional layer and train only final layer (classifier) in this experiment. For a high data regime, the performance of CNN initialized with edge detectors is worse than CNN initialized with random weights. Why do you think this happens? Answer this question in your submission of the written assignment.

Gemini Replied:

This phenomenon highlights a trade-off between **strong inductive bias** and **information preservation**.

Here is the reasoning you can use for your written assignment:

1. Feature Bottleneck / Information Loss The fixed **edge-detector kernels** are highly specific: they are designed to detect *only* horizontal or vertical edges. They effectively act as a narrow filter. If the dataset contains features they are not explicitly designed for (such as diagonal edges, specific noise patterns, or variations in thickness), these kernels may produce near-zero activations for those inputs. This destroys information before it ever reaches the classifier. The final layer cannot learn to classify data it cannot "see."

2. Random Projections Preserve Geometry Random weights, while not semantically meaningful, act as a random projection. In high-dimensional spaces, random projections tend to preserve the distances and geometry of the input data (related to the Johnson-Lindenstrauss lemma). This means that different inputs (e.g., a diagonal edge vs. noise) will produce distinct, albeit messy, activation patterns.

3. The Role of the "High Data Regime" In a **high data regime**, the trainable classifier (the final layer) has enough examples to learn how to decode these complex, messy patterns created by the random filters.

- **With Edge Detectors:** The classifier is limited by the missing information (the "blind spots" of the fixed filters).
- **With Random Weights:** The classifier receives a richer, albeit noisier, representation and has enough data to learn the decision boundaries necessary to separate the classes.

Summary The random weights provide a **richer representation** that preserves more information about the input, whereas the fixed edge detectors effectively "blind" the model to features they were not pre-programmed to recognize. With enough data, the classifier can solve the puzzle of random features, but it cannot recover the data discarded by the fixed filters.

The above analysis captures the point of the question based on the context of the previous code.

You asked:

```
def train_model(model, optimizer, num_epochs, train_loader, valid_loader):

    criterion=nn.CrossEntropyLoss()

    untrained_model = deepcopy(model)

    train_acc_list, valid_acc_list, train_loss_list, valid_loss_list = [], [], [] ,[]

    for epoch in tqdm(range(num_epochs)):
```

```
        train_loss, train_acc = train_one_epoch(model, optimizer, criterion, train_loader, device, epoch, verbose=False)

        valid_loss, valid_acc, confusion_matrix = evaluate(model, criterion, valid_loader, device, verbose=False)

        train_acc_list.append(train_acc)

        valid_acc_list.append(valid_acc)

        train_loss_list.append(train_loss)

        valid_loss_list.append(valid_loss)


    return {"final_valid_acc": valid_acc_list[-1], "train_acc": train_acc_list, "valid_acc": valid_acc_list,

            "train_loss": train_loss_list, "valid_loss": valid_loss_list,

            "confusion_matrix": confusion_matrix}



lr = 1e-2

num_epochs = 30

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

criterion = nn.CrossEntropyLoss()


cnn_acc_list = list()

mlp_acc_list = list()


cnn_kernel_dict = dict()

untrained_cnn_kernel_dict = dict()


for num_image, train_loader in train_loader_dict.items():

    print("Training with {} images".format(num_image))

    set_seed(seed)
```

```
    cnn_model = SimpleCNN(kernel_size=7)

    untrained_cnn_model = deepcopy(cnn_model)

    cnn_model.to(device)


    mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])

    mlp_model.to(device)


    mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)


    mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader, valid_loader)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, valid_loader)


    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"], mlp_results["train_loss"],
mlp_results["train_acc"])

    vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"], mlp_results["valid_loss"],
mlp_results["valid_acc"])


    print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"],
mlp_results["final_valid_acc"]))
```

Question

List every epochs that you trained the model. Final accuracy of CNN should be at least 95% for 30 images per class. Answer this question in your submission of the written assignment.

Question

Check the learned kernels. What do you observe? Answer this question in your submission of the written assignment.

Question (Optional)

You might find that with the high number of epochs, validation loss of MLP is increasing while validation accuracy increasing. How can we interpret this? Answer this question in your submission

of the written assignment.

(Hint: you may find papers that discuss calibrations related to this question (e.g., paper)

Question (Optional)

Do hyperparameter tuning. And list the best hyperparameter setting that you found and report the final accuracy of CNN and MLP. Answer this question in your submission of the written assignment.

> **Gemini Replied:**

I cannot execute the code directly to generate the training logs for you, but I can provide the answers and explanations you need for the written assignment based on deep learning theory and the specific design of this experiment.

Here are the responses for your submission, including a Python script to help you perform the hyperparameter tuning.

Answers for Written Assignment

Q: List every epochs that you trained the model. *(Since I cannot run the code, you should copy the output from your Jupyter Notebook here. It will look like a list from Epoch 0 to Epoch 29. Ensure that for the run with 30 images, your CNN accuracy is > 0.95).*

Q: Check the learned kernels. What do you observe? You should observe that the **learned kernels** in the CNN look remarkably similar to standard **edge detection filters** (like Sobel or Prewitt operators). Specifically:

1. **Structure:** You will see distinct light and dark bands (positive and negative weights) arranged horizontally or vertically.
2. **Function:** This indicates the model has learned that the most efficient way to minimize loss on this dataset is to detect local intensity gradients (edges).
3. **Comparison:** Unlike the "Untrainable" kernels (which look like random static/noise) or the MLP weights (which often look like unstructured noise or faint global templates), the CNN kernels are sharp, geometric, and interpretable.

Q: (Optional) You might find that with the high number of epochs, validation loss of MLP is increasing while validation accuracy increasing. How can we interpret this? This phenomenon is often linked to **model calibration** and **overconfidence** (discussed in papers like *On Calibration of Modern Neural Networks*, Guo et al., 2017).

1. **Overconfidence:** As training proceeds, the model continues to minimize the Cross-Entropy loss by pushing its predicted probabilities (logits) closer to 1.0 (for the predicted class) and 0.0 (for others).
2. **The Penalty:** Cross-Entropy loss is logarithmic. If the model makes a mistake on a validation image, but does so with *high confidence* (e.g., 99% sure it's "Vertical" when it's actually

"Horizontal"), the loss penalty is massive.

3. **The Result:** The model might correctly classify a few more difficult examples (increasing Accuracy slightly), but its increasing confidence on the few examples it still gets wrong drives the average Loss up significantly. The model is becoming "sharper" but not necessarily "wiser" regarding the distribution.

Hyperparameter Tuning Code

To answer the final optional question, I have generated a script that performs a grid search over Learning Rate and Hidden Dimensions. You can copy the best parameters found by this script into your assignment.

Even without screenshots, the answers appear reasonable.

Hyperparameter Tuning
Dec 3, 4:48 PM

Summary

1. **Epochs:** Copy your notebook output (0-29).
2. **Kernels:** They look like clean, directional edge detectors (Sobel-like), unlike the noisy/random weights of the MLP or untrained models.
3. **Loss vs Acc:** The model is becoming **overconfident**. It classifies more images correctly (Acc ↑), but minimizes the loss on correct images so aggressively that any mistake made with high confidence results in a massive penalty, driving the average Loss ↑.
4. **Tuning:** Use the `tuning_experiment.py` code above to perform a grid search. A likely "best" setting is often **LR=0.01** with **Momentum=0.9**, but larger hidden layers for the MLP (e.g., `[200, 100]`) may help it compete better with the CNN.

> **You asked:**

##

TODO: Try other num_epochs. Final accuracy of CNN should be at around

95-99% for 30 images per class.

##

##

END OF YOUR CODE

##

lr = 1e-2

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

criterion = nn.CrossEntropyLoss()


cnn_acc_list = list()

mlp_acc_list = list()


cnn_kernel_dict = dict()

untrained_cnn_kernel_dict = dict()


for num_image, train_loader in train_loader_dict.items():

    print("Training with {} images".format(num_image))

    set_seed(seed)

    cnn_model = SimpleCNN(kernel_size=7)

    untrained_cnn_model = deepcopy(cnn_model)

    cnn_model.to(device)


    mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])

    mlp_model.to(device)


    mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)


    mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader, valid_loader)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, valid_loader)


    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"], mlp_results["train_loss"],
mlp_results["train_acc"])
```

vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"], mlp_results["valid_loss"], mlp_results["valid_acc"])

cnn_kernel_dict[num_image] = deepcopy(cnn_model.conv1.weight.data.detach().cpu())

untrained_cnn_kernel_dict[num_image] = deepcopy(untrained_cnn_model.conv1.weight.data.detach().cpu())

print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"], mlp_results["final_valid_acc"]))

Gemini Replied:

I've updated `tuning_experiment.py` to include the main experimental loop you provided. I set `num_epochs = 30` as requested to target that >95% accuracy benchmark, and I included the necessary data setup code (creating `train_loader_dict`) so the script is self-contained and runnable (assuming the model classes and dataset class are imported).

In this script:

1. **`num_epochs`** is set to **30**.
2. It generates the **`train_loader_dict`** for 10, 30, 50, and 100 images.
3. It runs the **comparison loop** between the MLP and CNN.

You asked:

On this occasion, the code originally disobeyed the structure of the #TODO block, but fixed the solution when prompted to do so.

Please only provide code for the # TODO block.

Gemini Replied:

Based on the requirement to reach ~95-99% accuracy with 30 images per class, increasing the epochs slightly is usually the solution.

Python

```
################################################################################
# TODO: Try other num_epochs. Final accuracy of CNN should be at around       #
# 95-99% for 30 images per class.                                              #
################################################################################

num_epochs = 50

################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

You asked:

Question

How much more data is needed for MLP to get a competitive performance with CNN? Answer this question in your submission of the written assignment.

Gemini Replied:

Based on the results typically observed in this "Inductive Bias" experiment, here is the answer for your written assignment:

Answer:

MLPs require significantly more data—often by an order of magnitude (10x to 50x more)—to achieve performance competitive with a CNN.

Reasoning:

1. **Inductive Bias vs. Learning from Scratch:**
    - **CNNs** have a strong inductive bias (locality and translation invariance). They are pre-designed to understand that pixels close together are related and that an edge in the top-left corner looks the same as an edge in the bottom-right. They only need to see a few examples to learn the *shape* of an edge.
    - **MLPs** lack this bias. They treat every pixel as an independent input with no inherent spatial relationship to its neighbors. Consequently, an MLP must learn to recognize a "vertical edge" separately at every single possible location in the image 28x28 grid.
2. **Sample Complexity:**
    - Because the MLP has to learn these spatial invariances from raw data rather than having them hard-coded, its **sample complexity** is much higher.
    - While a CNN might solve the task with **30-50 images**, an MLP might need **500 to 1,000+ images** to begin approximating the same generalization performance, effectively "brute-

forcing" the problem by seeing every possible translation of the features.

I provided a copy/paste of the data to allow for more direct comparison rather than just speculating based on the setup. Gemini was able to accurately analyse this.

You asked:

Here is the training accuracy output you can use to answer this question. Training with 10 images100%|██████████| 50/50 [00:01<00:00, 30.19it/s]

100%|██████████| 50/50 [00:01<00:00, 29.08it/s]

CNN Acc: 48.666666666666664, MLP Acc: 40.0

Training with 30 images100%|██████████| 50/50 [00:02<00:00, 21.57it/s]

100%|██████████| 50/50 [00:02<00:00, 20.09it/s]

CNN Acc: 92.0, MLP Acc: 72.0

Training with 50 images100%|██████████| 50/50 [00:03<00:00, 16.62it/s]

100%|██████████| 50/50 [00:03<00:00, 15.33it/s]

CNN Acc: 98.0, MLP Acc: 84.66666666666667

Training with 100 images100%|██████████| 50/50 [00:04<00:00, 10.89it/s]

100%|██████████| 50/50 [00:05<00:00, 9.63it/s]

CNN Acc: 98.0, MLP Acc: 90.66666666666667

Gemini Replied:

Based on the specific training logs you provided, here is the answer for your written assignment.

Answer for Written Assignment

Observation: Based on the experimental results, the **MLP requires significantly more data (at least 3-4x more)** to achieve performance competitive with the CNN.

Evidence from the logs:

- **CNN Performance:** With just **30 images**, the CNN achieves an accuracy of **92.0%**. It effectively solves the task (reaching **98.0%**) with **50 images**.
- **MLP Performance:** With **30 images**, the MLP only reaches **72.0%**. To reach a comparable accuracy to the CNN's low-data performance (specifically, to cross the 90% threshold), the MLP required **100 images** (reaching **90.7%**).

Conclusion: The MLP needed **100 images** to achieve roughly the same accuracy (~90-92%) that the CNN achieved with only **30 images**. This demonstrates that the CNN is much more data-efficient due to its **inductive bias** (translation invariance and locality), which allows it to generalize features (edges) across the image without needing to see them in every possible location during training. The MLP, lacking this bias, must effectively "memorize" edges in different positions, requiring significantly more training examples to generalize.

> **You asked:**

I included a screenshot here, which was cut off, but the model was able to accurately summarize why the confusion matrix shows some misclassification based on the change to the training data.

Q4. Domain Shift between Training and Validation Set

In this problem, we will see how the model performance changes when the domain of the training set and that of the validation set are different. We will generate training set images with edges that locate only half of the image and validation set images with edges that locate only the other half of the image. Let's repeat the same experiment as the previous problem. In this example, you will see that both CNN and MLP performance are worse than those in the previous question. If two models learn how to extract edges, they should be able to classify the images with edges even though the edges locate in the other half of the images. However, both models suffer from performance degration (especially for MLP). What would be the problem? To investigate this, let's first look at the confusion matrices for both models link. Question

Why do you think the confusion matrix looks like this? Answer this question in your submission of the written assignment.

(Hint: Visualize some of the images in the training and validation set. And we are using kernel_size=7, which is large relative to the image size.)

> **Gemini Replied:**

Based on the experiment setup (Training on one half, Validation on the other) and the hint regarding the large kernel size (7×7), here is the explanation for the Confusion Matrix and performance degradation for your written assignment.

Answer for Written Assignment

Observation of Confusion Matrices:

- **MLP:** The confusion matrix likely shows the model predicting **"None" (No Edge)** or a single class for almost all validation images, regardless of the true label. The accuracy is likely near random chance or the baseline probability of the "None" class.
- **CNN:** The confusion matrix likely shows some correct predictions but a significant number of errors where edges are misclassified as "None" or other orientations, despite the model's theoretical translation invariance.

Why does this happen?

1. **MLP (Lack of Translation Invariance):**
    - **Spatial Overfitting:** MLPs treat every pixel as a unique, independent input feature. The weights connecting to the pixels in the "training half" of the image are optimized to detect edges. However, the weights connecting to the "validation half" never see an edge during training—they only see black background.
    - **Result:** When an edge appears in the "validation half," it is multiplied by these **untrained, random weights**. This produces a noise activation that does not trigger the "Vertical" or "Horizontal" class output, causing the model to default to "None" or guess randomly. The MLP has no mechanism to transfer knowledge from the left side to the right side.
2. **CNN (Context Overfitting with Large Kernels):**
    - **Boundary Effects:** While CNNs are translation invariant, the **large kernel size (7×7)** relative to the image size (28×28) means the filter "sees" a lot of context around the edge.
    - **Context Mismatch:** In the training set (e.g., left half), a vertical edge might often appear adjacent to the left zero-padding. The learned filter effectively memorizes **"Edge + Left Padding"**. In the validation set (right half), the vertical edge appears adjacent to the *right* zero-padding or empty space on its left.
    - **Failure:** The filter, having overfitted to the specific *surrounding context* of the training edges, fails to activate strongly when that context changes (the domain shift), even though the edge itself is identical. A smaller kernel (e.g., 3×3) would likely suffer less from this issue as it captures less background context.

> **You asked:**

We can do better than this. We didn't explore hyperparameter space yet. Let's search hyperparameters that can generalize well to the validation set. We will change the learning rate, the number of epochs, and kernel size for CNN.

##

```
# TODO: Try other num_epochs, lr, kernel_size. The validation accuracy      #
# should achieve around 97-100% for 10 images per class.                    #
################################################################################
################################################################################
#                          END OF YOUR CODE                                    #
################################################################################
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, valid_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"], None, None)
```

```
vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"], None, None)


print("CNN Acc: {}".format(cnn_results["final_valid_acc"]))
```

Gemini Replied:

To achieve 97-100% accuracy with only **10 images per class**, we need to rely heavily on the CNN's inductive bias.

- `kernel_size`: Reducing this to **3** (from 7) is the most critical change. A 3x3 kernel forces the model to look at very local features (edges) rather than memorizing larger patches of the background context, which helps it generalize from very few examples.
- `num_epochs`: Increasing this to **50** or **100** allows the model to fully converge, as each epoch is very short with such a small dataset.

Here is the code for the TODO block:

The original code provided by Gemini failed to reach the accuracy target of 97%. When prompted, the model was able to recognize that a greater number of epochs should be used and suggested 1000, which worked.

Hyperparameter Tuning
Dec 3, 5:32 PM

You asked:

This only got to CNN Acc: 66.0 and we want higher (97-100%). What should I change? Should I increase num_epochs significantly?

Gemini Replied:

I have updated the **Canvas** file `Hyperparameter Tuning` to increase the number of epochs to 1000.

Why this change is needed: You are training on a tiny dataset (10 images). In deep learning, "epochs" refers to passes over the dataset. With only 10 images, 50 epochs is extremely few total gradient updates (steps). The model simply hasn't had enough iterations to converge. Increasing to **1000 epochs** provides enough steps for the optimizer to find the correct weights, while the small **kernel_size=3** ensures those weights generalize correctly to the other half of the image. I have updated the **Canvas** file `Hyperparameter Tuning` to increase the number of epochs to 1000.

Why this change is needed: You are training on a tiny dataset (10 images). In deep learning, "epochs" refers to passes over the dataset. With only 10 images, 50 epochs is extremely few total gradient updates (steps). The model simply hasn't had enough iterations to converge. Increasing to **1000 epochs** provides enough steps for the optimizer to find the correct weights, while the small **kernel_size=3** ensures those weights generalize correctly to the other half of the image.

You asked:

Question

Why do you think MLP fails to learn the task while CNN can learn the task? Answer this question in your submission of the written assignment.

(Hint: Think about the model architecture.)

Gemini is able to recall the setup for this new variation of the problem from the original ipynb.

Gemini Replied:

Here is the answer for your written assignment regarding the failure of the MLP compared to the CNN in the domain shift task.

Answer for Written Assignment

The core reason is the difference in Inductive Bias, specifically Translation Invariance.

1. **MLP (Spatial Dependence):**
   - **Architecture:** An MLP is a fully connected network where every input pixel has its own unique weight connecting it to the hidden layer.
   - **The Failure:** Because weights are tied to specific pixel coordinates, the MLP learns to recognize edges **only at the specific locations** where they appeared in the training set (e.g., the left half).
   - **Result:** When an edge appears in the validation set (the right half), the MLP processes it using weights that were trained on empty space (background). The model has no mechanism to "know" that the pattern it learned on the left is the same concept as the pattern on the right. It essentially has to relearn the concept of an "edge" for every single pixel location.
2. **CNN (Translation Invariance):**
   - **Architecture:** A CNN uses **weight sharing** via convolutional kernels. The same small filter (e.g., 3×3) is slid across the entire image.
   - **The Success:** Once the CNN learns a filter that detects a vertical edge in the top-left corner, that same filter is mathematically applied to the bottom-right corner (and everywhere else).
   - **Result:** Even though the model never saw an edge on the right side during training, the shared filter successfully "fires" when it encounters the edge in the validation set, allowing it to generalize perfectly to the domain shift.

You asked:

Q5. When is CNN Worse than MLP?

In this problem, we will see that CNN is not always better than MLP in the image domain. Using CNN assumes that the data has locally correlated, whatever data looks. We can manually 'whiten' or remove such local correlation simply by applying random permutation to the images. A random permutation matrix is a matrix that has the same number of rows and columns. Each row and column has the same number of 1s. The rest of the elements are 0s. For example, the following is a random permutation matrix.

[[0, 1, 0, 0],

[0, 0, 0, 1],

[1, 0, 0, 0],

[0, 0, 1, 0]]

This matrix randomly reorders the elements of the vector. For example, if we apply this matrix to the vector [1, 2, 3, 4], we will get [2, 4, 1, 3]. If we apply this matrix to the image, we will get the image with the same content, but the pixels are randomly shuffled. One property of the random permutation matrix is that it is invertible. It means that we can recover the original image by simply applying the inverse matrix to the shuffled image. From the information-theoretical perspective, the random permutation matrix preserves the mutual information of the image and the label.

We will repeat the same experiment as the previous problem. Visualize the dataset first.

Now let's train CNN and MLP on the permuted dataset.

Question

What do you observe? What is the reason that CNN is worse than MLP? Answer this question in your submission of the written assignment.

(Hint: Think about the model architecture.)

Question

Assuming we are decreasing kernel size of CNN. Does the validation accuracy increase or decrease? Why? Answer this question in your submission of the written assignment.

> **Gemini Replied:**

Here are the answers for your written assignment regarding the random permutation experiment.

Answer for Written Assignment

Question 1: What do you observe? What is the reason that CNN is worse than MLP?

- **Observation:** The **MLP maintains its performance** (achieving roughly the same accuracy as on the un-permuted data), whereas the **CNN's performance collapses** (often dropping to near random chance or performing significantly worse than the MLP).

- **Reason:** This happens because **random permutation destroys local spatial correlations**, which are the fundamental assumption of CNNs.
    - **CNN Failure:** CNNs rely on **locality**—the assumption that pixels next to each other form meaningful structures (like lines or curves). When pixels are shuffled, a 3×3 convolutional kernel "sees" a group of random, unrelated pixels that have no geometric relationship. The inductive bias of the CNN (looking for local features) becomes a liability because local neighborhoods no longer contain useful information.
    - **MLP Success:** MLPs are **permutation invariant** regarding the input ordering, provided the permutation is fixed. Since the MLP treats every pixel as an independent feature with a dedicated weight, shuffling the pixels simply means the model learns the same weights but in a different order. It does not rely on neighbor relationships, so it can solve the puzzle just as easily as the original image.

Question 2: Assuming we are decreasing kernel size of CNN. Does the validation accuracy increase or decrease? Why?

- **Result:** The validation accuracy will likely **decrease** (or remain effectively at random chance).
- **Reason:** In the permuted image, the "edge" information is shattered and scattered across the entire image. To recognize the edge, the model needs to correlate pixels that are now far apart in the grid.
    - **Receptive Field:** A CNN with a small kernel size has a very small **effective receptive field**. It only looks at immediate neighbors. Since the meaningful correlations are now global (long-range), a small kernel cannot capture the relationship between the scattered pixels necessary to identify the edge.
    - To have any chance of learning on permuted data, a CNN would need a kernel size (or depth) large enough to cover the entire image (effectively becoming a fully connected layer), so reducing the kernel size moves the architecture further away from the solution.

> **You asked:**

Again, the screenshot I included was cut off, but Gemini was able to synthesies the permuted data experiment filters.

Question

How do the learned kernels look like? Explain why. Answer this question in your submission of the written assignment.

From the above example, we can see that CNN is not always better than MLP. We have to think about the domain (or task) of the dataset and the model architecture to decide which model is better.

Gemini Replied:

Here is the answer for your written assignment regarding the learned kernels in the **permuted data** experiment.

Answer for Written Assignment

Observation: The learned kernels will look like **random, unstructured noise**. Unlike the clean, geometric edge detectors (Sobel-like filters) seen in the previous experiments, these kernels will have no discernible patterns (no clear lines, bands, or gradients).

Reasoning:

1. **Destruction of Locality:** The random permutation process destroys the local spatial correlations that CNNs rely on. In the original images, an "edge" is defined by a line of neighboring pixels with contrasting intensities. In the permuted images, the pixels forming that edge are scattered randomly across the 28x28 grid.

2. **Kernel Failure:** A convolutional kernel (e.g., 3×3) only looks at a tiny, local neighborhood. Because of the permutation, the pixels inside this 3×3 window are now completely unrelated to each other semantically. The kernel cannot find any consistent local pattern (like a vertical line) to detect.

3. **Optimization:** Since there are no local features to learn, the gradient updates effectively push the weights in random directions trying to minimize loss, resulting in kernels that look like static noise. The CNN fails because its **inductive bias** (assuming locality) is now mismatched with the data structure (which requires global correlation).

You asked:

Question

Compare the performance of CNN with max pooling and average pooling. What are the advantages of each pooling method? Answer this question in your submission of the written assignment.

CNN-maxpool Acc: 99.6, CNN-avgpool Acc: 96.4

Gemini Replied:

I can help you answer that question for your assignment. Based on the results you provided (Max Pooling: 99.6% vs. Average Pooling: 96.4%), here is the comparison and explanation.

Answer for Written Assignment

Observation: Max pooling outperforms average pooling on this edge detection task (99.6% vs 96.4%).

Comparison & Advantages:

1. **Max Pooling (The Winner here):**
    - **Mechanism:** Selects the maximum value in the pooling window.
    - **Advantage:** It acts as a **feature selector**. It preserves the strongest signal (the sharpest edge detection) and discards the weaker signals.
    - **Why it works here:** Edge detection relies on identifying high-frequency features (sharp transitions). Max pooling preserves the high activation of a detected edge while ignoring the surrounding low-activation background (black pixels). This keeps the signal strong and distinct.
2. **Average Pooling:**
    - **Mechanism:** Calculates the average value of all pixels in the pooling window.
    - **Advantage:** It effectively performs **smoothing** (low-pass filtering). It is useful for tasks where the *amount* of a feature in a region matters more than its precise location or peak intensity (e.g., global average pooling in ResNets or texture analysis).
    - **Why it failed here:** In this task, averaging **dilutes the signal**. If a window contains a sharp edge (high value) and mostly background (zeros), averaging them together results in a much lower value. This makes the edge "blurry" to the next layer and harder to distinguish from noise, lowering accuracy.

> ### You asked:

Q7. Wider/Deeper CNNs

Can we further improve the performance by making the architecture deeper and wider? In this question, we focus on the dataset where there are only 30 images per class and try to push the performance of the CNNs further.

The patterns that we have to detect are 5 but our kernels per layer (num_filters in the network definition above) are only 3. Intuitively, this is quite a suboptimal. Here, we will investigate the affect of increasing width and depth. Let's use the same dataset but we will use DeeperCNN and WiderCNN in cnn.py. DeeperCNN has 2 times more layers than SimpleCNN and WiderCNN has 2 times more kernels per layer than SimpleCNN. Let's train the models and visualize the validation accuracy.

##

```
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs,   #
# lr, kernel_size. Also try a different optimizer (e.g., Adam)              #
# The validation accuracy can reach above 98% for 30 images per class.      #
################################################################################
################################################################################
#                          END OF YOUR CODE                                    #
################################################################################
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[30]
set_seed(seed)
deeper_cnn_model = DeeperCNN(kernel_size=kernel_size)
untrained_deeper_cnn_model = deepcopy(deeper_cnn_model)
deeper_cnn_model.to(device)

deeper_cnn_optimizer = optim.SGD(deeper_cnn_model.parameters(), lr=lr, momentum=0.9)
# deeper_cnn_optimizer = optim.Adam(deeper_cnn_model.parameters(), lr=lr)  # try me!

deeper_cnn_results = train_model(deeper_cnn_model, deeper_cnn_optimizer, num_epochs,
train_loader, valid_loader)

vis_training_curve(deeper_cnn_results["train_loss"], deeper_cnn_results["train_acc"], None, None)
vis_validation_curve(deeper_cnn_results["valid_loss"], deeper_cnn_results["valid_acc"], None,
None)

print("CNN Acc: {}".format(deeper_cnn_results["final_valid_acc"]))
```

The provided code here worked one-shot!

Gemini Replied:

I've updated the **Canvas** file `Hyperparameter Tuning` (which I've renamed to `tuning_solution.py` to match your context) with the configuration for the Deeper and Wider CNN experiments.

Changes made:

- **Optimizer:** Switched to `optim.Adam` as hinted. Adam often converges faster and more reliably on these types of problems compared to SGD.
- **Hyperparameters:**
  - `num_epochs = 500`: Ensuring sufficient convergence time for the deeper/wider networks.
  - `lr = 1e-3`: A standard starting point for Adam.
  - `kernel_size = 3`: Keeping the small kernel size to preserve the locality benefit we discovered in the previous steps.
- **Architecture:** Added the setup for both `DeeperCNN` and `WiderCNN` to allow for a direct comparison on the 30-image dataset.

You asked:

```
################################################################################
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs,   #
# lr, kernel_size. Also try a different optimizer (e.g., Adam)              #
# The validation accuracy can reach above 98% for 30 images per class.      #
################################################################################


################################################################################
#                              END OF YOUR CODE                              #
################################################################################
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()


train_loader = train_loader_dict[30]
```

```
set_seed(seed)

wider_cnn_model = WiderCNN(kernel_size=kernel_size)

untrained_wider_cnn_model = deepcopy(wider_cnn_model)

wider_cnn_model.to(device)


wider_cnn_optimizer = optim.SGD(wider_cnn_model.parameters(), lr=lr, momentum=0.9)

# wider_cnn_optimizer = optim.Adam(wider_cnn_model.parameters(), lr=lr)  # try me!


wider_cnn_results = train_model(wider_cnn_model, wider_cnn_optimizer, num_epochs, train_loader,
valid_loader)


vis_training_curve(wider_cnn_results["train_loss"], wider_cnn_results["train_acc"], None, None)

vis_validation_curve(wider_cnn_results["valid_loss"], wider_cnn_results["valid_acc"], None, None)


print("CNN Acc: {}".format(wider_cnn_results["final_valid_acc"]))
```

Again, the provided code worked on the first try and appears to have fulfilled the requirements set out in the ipynb file.

Gemini Replied:

I've updated the **Canvas** file `Hyperparameter Tuning` to focus on the `WiderCNN` experiment, incorporating the code snippet you provided. I also configured the hyperparameters (epochs, learning rate, and optimizer) to ensure the model achieves the target accuracy (>98%) on the 30-image dataset.

Changes:

- **Targeted WiderCNN:** The script now runs the specific `WiderCNN` training loop.
- **Optimizer Switch:** Uncommented and enabled `optim.Adam` as it generally converges faster than SGD for this task.
- **Hyperparameters:** Set `num_epochs = 500` and `lr = 1e-3` to ensure convergence.
- **Kernel Size:** Maintained `kernel_size = 3` to leverage locality.

small_dataset_generation.py

```
################################################################################
# TODO: Generate dataset with 10 images per class                              #
# Hint: Refer visualize_data_config and use EdgeDetectionDataset and           #
# transforms function provided above.                                          #
################################################################################

# We use explicit class definitions to ensure exactly 10 samples per class.
small_dataset_config = {
    'data_per_class': 10,
    'num_classes': 3,
    'class_type': ['horizontal', 'vertical', 'none'],
    'img_size': 28,
    'min_noise': 0.0,
    'max_noise': 0.1,
    'min_edge_contrast': 0.5,
    'max_edge_contrast': 1.0
}

small_dataset = EdgeDetectionDataset(small_dataset_config, transform=transforms)

################################################################################
#                                END OF YOUR CODE                              #
################################################################################

small_dataset_loader = None

################################################################################
# TODO: Implement dataloader                                                   #
# Hint: You should flag shuffle = True for training data loader                #
# This flag makes huge difference in training                                  #
################################################################################

# Calculate batch size based on total samples (10 * 3 = 30)
# We import DataLoader just in case it wasn't in the global scope.
from torch.utils.data import DataLoader

small_dataset_loader = DataLoader(small_dataset, batch_size=30, shuffle=True)

################################################################################
#                                END OF YOUR CODE                              #
################################################################################
```

Dataset Sweep

```
import torch
import torchvision.transforms as T
from torch.utils.data import DataLoader

# Assuming EdgeDetectionDataset is available in the environment
# from edge_detection_dataset import EdgeDetectionDataset

################################################################################
# Setup Configuration
################################################################################
# Initialize variables as per the notebook context
train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
valid_batch_size = 256

################################################################################
# TODO: Implement train_loader_dict for each number of training images.     #
# Key: The number of training images (5, 10, 30, 50, and 100)               #
# Value: The corresponding dataloader                                       #
# The validation set size is 50 images per class                            #
################################################################################

# 1. Create the Validation Loader (50 images per class)
valid_dataset_config = {
    'data_per_class': 50,
    'img_size': 28,
    'min_noise': 0.0,
    'max_noise': 0.1,
    'min_edge_contrast': 0.5,
    'max_edge_contrast': 1.0
}

# Added 'valid' mode argument to the constructor
```

```
valid_dataset = EdgeDetectionDataset(valid_dataset_config, 'valid',
transform=transforms)
valid_loader = DataLoader(valid_dataset, batch_size=valid_batch_size, shuffle=False)


# 2. Create Train Loaders for each size in the list
for n_images in num_images_list:
    train_dataset_config = {
        'data_per_class': n_images,
        'img_size': 28,
        'min_noise': 0.0,
        'max_noise': 0.1,
        'min_edge_contrast': 0.5,
        'max_edge_contrast': 1.0
    }

    # Added 'train' mode argument to the constructor
    train_dataset = EdgeDetectionDataset(train_dataset_config, 'train',
transform=transforms)

    train_loader_dict[n_images] = DataLoader(train_dataset,
batch_size=train_batch_size, shuffle=True)

################################################################################
#                               END OF YOUR CODE                               #
################################################################################
```

Hyperparameter Tuning

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as T
from torch.utils.data import DataLoader
from copy import deepcopy

# Placeholder imports - ensure these classes are defined in your notebook/environment
# from your_notebook import EdgeDetectionDataset, SimpleCNN, ThreeLayerMLP, train_model
# from your_notebook import vis_training_curve, vis_validation_curve, set_seed

def run_comparison_experiment():
    # ----------------------------------------------------------
    # 1. Data Setup (Recreating train_loader_dict for 10, 30, 50, 100 images)
    # ----------------------------------------------------------
    print("Setting up data for experiment...")
    transforms = T.Compose([T.ToTensor()])

    num_images_list = [10, 30, 50, 100]
    train_loader_dict = {}

    # Validation Loader (Fixed 50 images per class)
    valid_config = {
        'data_per_class': 50,
        'img_size': 28,
        'min_noise': 0.0, 'max_noise': 0.1,
        'min_edge_contrast': 0.5, 'max_edge_contrast': 1.0
    }
    # valid_dataset = EdgeDetectionDataset(valid_config, 'valid', transform=transforms)
    # valid_loader = DataLoader(valid_dataset, batch_size=256, shuffle=False)

    # Train Loaders
    for n_images in num_images_list:
        train_config = {
            'data_per_class': n_images,
            'img_size': 28,
            'min_noise': 0.0, 'max_noise': 0.1,
            'min_edge_contrast': 0.5, 'max_edge_contrast': 1.0
```

```
        }
        # train_dataset = EdgeDetectionDataset(train_config, 'train',
transform=transforms)
        # train_loader_dict[n_images] = DataLoader(train_dataset, batch_size=10,
shuffle=True)

    # NOTE: Uncomment the dataset creation lines above if running standalone.
    # For now, we assume train_loader_dict and valid_loader are passed or available.

    # ------------------------------------------------------------
    # 2. Experiment Loop
    # ------------------------------------------------------------

    # TODO: Try other num_epochs. Final accuracy of CNN should be at around
    # 95-99% for 30 images per class.
    num_epochs = 30

    lr = 1e-2
    seed = 42 # Ensure seed is defined
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    criterion = nn.CrossEntropyLoss()

    cnn_acc_list = list()
    mlp_acc_list = list()

    cnn_kernel_dict = dict()
    untrained_cnn_kernel_dict = dict()

    # Iterate through the dictionary of loaders
    # We assume train_loader_dict is populated in the actual notebook execution
    for num_image, train_loader in train_loader_dict.items():
        print("Training with {} images".format(num_image))
        # set_seed(seed) # Call your seed function here

        # Initialize Models
        cnn_model = SimpleCNN(kernel_size=7)
        untrained_cnn_model = deepcopy(cnn_model)
        cnn_model.to(device)

        mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])
```

```
        mlp_model.to(device)

        # Optimizers
        mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)
        cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

        # Train
        mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader,
valid_loader)
        cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader,
valid_loader)

        # Visualization (Assuming these functions exist in your notebook)
        # vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
mlp_results["train_loss"], mlp_results["train_acc"])
        # vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"],
mlp_results["valid_loss"], mlp_results["valid_acc"])

        # Store Kernels for analysis
        if hasattr(cnn_model, 'conv1'):
            cnn_kernel_dict[num_image] =
deepcopy(cnn_model.conv1.weight.data.detach().cpu())
            untrained_cnn_kernel_dict[num_image] =
deepcopy(untrained_cnn_model.conv1.weight.data.detach().cpu())

        print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"],
mlp_results["final_valid_acc"]))

        cnn_acc_list.append(cnn_results["final_valid_acc"])
        mlp_acc_list.append(mlp_results["final_valid_acc"])

if __name__ == "__main__":
    try:
        run_comparison_experiment()
    except NameError as e:
        print("Error: Ensure models (SimpleCNN, ThreeLayerMLP) and utils
(EdgeDetectionDataset, train_model) are defined.")
        print(f"Details: {e}")
```